# PYTHON 基础与进阶面试笔记

这份笔记在你原始内容的基础上做了三件事：

1. 补全容易被追问的细节。
2. 修正少量不严谨或容易引起误解的说法。
3. 按面试常见模块重新整理，便于背诵和临场展开。

## 0. PYTHON 语言特性总览

Python 通常被描述为：

- 解释型语言：代码通常由解释器逐步执行，而不是像 C/C++ 那样先整体编译成机器码再运行。
- 动态类型语言：变量在运行时绑定对象，对象自带类型，变量本身没有固定类型。
- 强类型语言：不同类型之间通常不会无声地自动转换，例如 `"1" + 1` 会直接报错，因此严格来说不应称为“弱类型语言”。
- 高级语言：屏蔽了大量底层内存管理细节，开发效率高。
- 多范式语言：支持面向对象、函数式、过程式等多种编程风格。
- 胶水语言：擅长把不同系统、库和服务连接起来。

一句常见的总结方式：

Python 适合快速开发、脚本自动化、数据分析、AI、后端服务和运维场景，但在纯 CPU 密集计算下通常不如 C/C++、Rust、Go 这类语言高效。

# 1. 元组和列表的区别

## 1.1 可变性

- `list` 是可变对象，可以增删改元素。
- `tuple` 是不可变对象，创建后不能修改其元素引用。

注意：

- “元组不可变”指的是元组内部保存的引用不可变。
- 如果元组里的元素本身是可变对象，例如列表，那么这个列表内部仍然可以变。

示例：

```
t = ([1, 2], 3)
t[0].append(4)    # 合法
# t[1] = 5        # 非法，TypeError
```

## 1.2 语法

- 列表：`[1, 2, 3]`
- 元组：`(1, 2, 3)`

特别注意单元素元组：

```
t = (1,)    # 必须带逗号
```

## 1.3 性能和内存

- 元组通常比列表更轻量。
- 元组不可变，解释器可做更多优化。
- 列表为了支持扩容，会有额外容量冗余。

## 1.4 哈希能力

- 元组只有在“所有元素都可哈希”时才可哈希。
- 列表一定不可哈希，因此不能作为字典键或集合元素。

## 1.5 使用场景

- 列表适合频繁修改的数据。
- 元组适合表达“固定结构的数据记录”，例如坐标、数据库返回的一行、函数多返回值。

# 2. 列表的底层内存管理

CPython 中，列表本质上更接近“动态数组”，内部保存的是对象引用，而不是对象本体。

## 2.1 内存布局

- 列表底层是一段连续的引用数组。
- 每个元素位置存放的是对象指针。
- 因此列表可以存放不同类型对象，因为存的是引用。

## 2.2 扩容机制

- 列表不是每插入一个元素就重新申请一次内存。
- 当容量不足时，会按一定策略“过量分配”。
- 这样能减少频繁扩容带来的拷贝成本。

常见面试说法：

- `append()` 的均摊时间复杂度是 `O(1)`。
- 但单次触发扩容时，拷贝成本可能是 `O(n)`。

## 2.3 删除和缩容

- 删除元素后，列表长度会变化。
- 但底层空间不一定立即归还给操作系统。
- CPython 在某些场景会缩容，但不会每次删除都缩。

## 2.4 时间复杂度

- 尾部追加：均摊 `O(1)`
- 尾部弹出：`O(1)`
- 按索引访问：`O(1)`
- 中间插入/删除：`O(n)`
- 成员判断：`O(n)`

# 3. 字符串插值与格式化

Python 常见字符串格式化方式有四种：

## 3.1 `%` 格式化

```
name = "Tom"
age = 18
print("name=%s age=%d" % (name, age))
```

特点：

- 语法老，但仍常见于旧项目。
- 对复杂对象格式化能力一般。

## 3.2 `str.format()`

```
print("name={} age={}".format(name, age))
print("name={name} age={age}".format(name="Tom", age=18))
```

特点：

- 支持位置参数、关键字参数、格式控制。

## 3.3 f-string

```
print(f"name={name} age={age}")
```

特点：

- Python 3.6+ 常用方式。
- 可读性最好，执行效率通常也较高。

## 3.4 `string.Template`

```
from string import Template
t = Template("hello, $name")
print(t.substitute(name="Tom"))
```

特点：

- 使用 `$name` 占位。
- 更适合简单模板替换。
- 常用于用户可控模板场景。

注意：

- 方法名是 `substitute()`，不是 `subtitle()`。
- 它不是"防止代码注入"的通用安全方案，只是语法更简单，不会像 f-string 那样执行表达式。

# 4. `MAP`、`FILTER`、`RANGE`

## 4.1 `map`

对可迭代对象中的每个元素应用一个函数，并返回一个迭代器。

```
nums = [1, 2, 3]
it = map(lambda x: x * 2, nums)
print(list(it))  # [2, 4, 6]
```

## 4.2 `filter`

根据函数返回值是否为真来筛选元素，并返回一个迭代器。

```
nums = [1, 2, 3, 4]
it = filter(lambda x: x % 2 == 1, nums)
print(list(it))  # [1, 3]
```

## 4.3 `range`

- `range()` 返回的不是列表。
- 它返回一个 `range` 对象，是一个可迭代对象，不是迭代器本身。
- 需要 `iter(range_obj)` 后才得到真正的迭代器。

```
r = range(1, 10, 2)
print(list(r))  # [1, 3, 5, 7, 9]
```

# 5. 闭包、装饰器、`GLOBAL`、`NONLOCAL`

## 5.1 闭包

闭包的本质：

- 内部函数引用了外部函数作用域中的变量。
- 外部函数返回内部函数后，这些变量仍然被保存。

```
def outer(x):
    def inner(y):
        return x + y
    return inner
```

## 5.2 `nonlocal`

如果要在内部函数中修改外层局部变量，需要使用 `nonlocal` 。

```
def outer():
    count = 0
    def inner():
        nonlocal count
        count += 1
        return count
    return inner
```

## 5.3 装饰器

装饰器本质是：

- 接收函数。
- 返回增强后的新函数。
- 在不修改原函数源码的前提下扩展行为。

常见用途：

- 日志
- 权限校验

- 缓存
- 统计耗时
- 重试

## 5.4 `functools.wraps`

如果不用 `wraps`，被装饰函数的 `__name__`、`__doc__` 等元信息会丢失。

```
from functools import wraps

def deco(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
```

## 5.5 `global`

在函数内部给全局变量重新赋值时，需要 `global`。

```
x = 10

def foo():
    global x
    x = 20
```

# 6. `is` 和 `==`

- `is` 比较对象身份，即是否是同一个对象。
- `==` 比较对象值是否相等。

```
a = [1, 2]
b = [1, 2]
print(a == b)  # True
print(a is b)  # False
```

注意：

- 不要用 `is` 比较普通整数、字符串值是否相等。
- 只有判断单例对象时常用 `is`，例如 `x is None`。

# 7. 类、实例方法、静态方法、类方法、元类

## 7.1 实例方法

- 默认第一个参数是 `self`。
- 访问实例属性和实例状态。

## 7.2 类方法

- 使用 `@classmethod`。
- 第一个参数是 `cls`。
- 常用于工厂方法、类级配置。

## 7.3 静态方法

- 使用 `@staticmethod`。
- 不依赖 `self` 或 `cls`。
- 更像放在类命名空间中的普通函数。

## 7.4 `__new__` 和 `__init__`

- `__new__` 负责"创建对象"，返回实例。
- `__init__` 负责"初始化对象"，不返回实例。

## 7.5 `type`

`type` 有两层含义：

- `type(obj)`：查看对象类型。
- `type(name, bases, namespace)`：动态创建类。

## 7.6 元类

元类就是"创建类的类"，默认元类是 `type`。

常见面试回答：

- 类是对象。
- 类由元类创建。
- 自定义元类常用于框架层，如 ORM、注册器、类校验。

# 8. MRO 和 `SUPER`

## 8.1 MRO

MRO 是方法解析顺序，决定多继承场景下方法查找路径。

Python 采用 C3 线性化算法。

```
print(MyClass.__mro__)
```

## 8.2 `super`

- `super()` 并不是"直接调用父类"那么简单。
- 它会按 MRO 顺序继续向后查找。
- 因此在多继承里尤其重要。

# 9. 迭代器、可迭代对象、生成器

## 9.1 可迭代对象

实现了 `__iter__()` 的对象通常可迭代，例如列表、元组、字典、字符串、集合、`range`。

## 9.2 迭代器

实现了：

- `__iter__()`
- `__next__()`

迭代器调用 `next()` 时不断返回下一个值，结束时抛出 `StopIteration`。

## 9.3 生成器

生成器是特殊的迭代器，通常通过两种方式创建：

- 在函数中使用 `yield`
- 生成器表达式：`(x * 2 for x in nums)`

特点：

- 惰性求值
- 节省内存

- 自动实现迭代器协议
- 可暂停和恢复执行

## 9.4 `yield` 和 `return`

- 生成器函数中执行 `yield` 会暂停。
- 执行 `return` 会终止生成器，并触发 `StopIteration` 。

## 9.5 生成器切片

生成器不能像列表那样直接切片，可使用 `itertools.islice()` 。

# 10. `WITH` 和上下文管理器

`with` 用于管理资源的获取和释放，核心是上下文管理协议：

- `__enter__()`
- `__exit__(exc_type, exc_value, traceback)`

用途：

- 文件操作
- 数据库连接
- 网络连接
- 锁管理

`__exit__()` 返回 `True` 表示吞掉异常，返回 `False` 或 `None` 表示异常继续向外抛。

# 11. `ANY`、`ALL`、断言、`HELP`、`DIR`

## 11.1 `any`

任意元素为真，则返回 `True`。

## 11.2 `all`

全部元素为真，才返回 `True`。

## 11.3 `assert`

断言常用于开发阶段的防御性检查：

```
assert x > 0, "x must be positive"
```

注意：

- 生产环境中不要把关键业务校验只写成 `assert`。
- 使用 `python -O` 可关闭断言。

## 11.4 `help` 和 `dir`

- `help(obj)`：查看帮助文档和 docstring。
- `dir(obj)`：查看对象可访问属性和方法。

# 12. 深拷贝、浅拷贝、参数传递

## 12.1 浅拷贝

只复制最外层对象，内部嵌套对象仍共享。

## 12.2 深拷贝

递归复制整个对象结构，内部对象也复制一份。

```
import copy
copy.copy(obj)
copy.deepcopy(obj)
```

## 12.3 参数传递

更严谨的说法是：

Python 采用“对象引用传递”或“共享传参”。

这意味着：

- 函数拿到的是对象引用。
- 重新绑定参数名不影响外部变量。
- 原地修改可变对象会影响外部对象。

# 13. 字符串常用方法

常见高频方法：

- `count()`
- `find()` / `index()`
- `replace()`
- `split()` / `join()`

- `strip()` / `lstrip()` / `rstrip()`
- `startswith()` / `endswith()`
- `lower()` / `upper()` / `title()`
- `isdigit()` / `isalpha()` / `isalnum()`
- 切片反转：`s[::-1]`

# 14. 字典和 `ORDEREDDICT`

## 14.1 dict

- 字典底层基于哈希表。
- 键必须可哈希。
- 平均情况下查找、插入、删除时间复杂度是 `O(1)`。

## 14.2 有序性

从 Python 3.7 开始，`dict` 的插入顺序成为语言规范。

因此现代 Python 中：

- 大多数"只需要保持插入顺序"的场景，直接用 `dict` 即可。

## 14.3 `OrderedDict`

`OrderedDict` 仍然有价值，主要因为：

- 提供 `move_to_end()`
- 提供更明确的顺序语义
- 适合实现 LRU 等需要频繁调整顺序的结构

注意：

- 现代 CPython 的 `dict` 不是“额外维护一个列表”来记录顺序，而是使用了更紧凑的哈希表布局。

# 15. JSON 序列化

常见方法：

```
import json
json.dumps(data, ensure_ascii=False)
json.loads(text)
```

自定义类型可以重写 `JSONEncoder.default()`：

```
import json

class MyEncoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, set):
            return list(obj)
        return super().default(obj)
```

# 16. 异常处理

完整结构：

```
try:
    ...
except ValueError:
    ...
else:
    ...
finally:
    ...
```

说明：

- `except`：捕获异常
- `else`：没有异常时执行
- `finally`：无论是否异常都执行

实践建议：

- 不要直接裸写 `except:`
- 优先捕获具体异常类型
- 异常信息要带上下文

# 17. 内存管理和垃圾回收

Python 内存管理主要包括：

## 17.1 引用计数

- 每个对象维护引用计数。
- 引用数为 0 时，可立即回收。
- 优点是回收及时。
- 缺点是无法单独处理循环引用。

## 17.2 垃圾回收器

为了解决循环引用，CPython 还引入了垃圾回收机制。

常见面试说法：

- 分代回收
- 标记可达对象

- 清理不可达对象

## 17.3 分代思想

经典理解：

- 新对象更容易死亡。
- 老对象更可能长期存活。
- 因此不同代的回收频率不同。

## 17.4 小对象内存池

CPython 对小对象使用专门分配器，减少频繁 `malloc/free` 带来的开销和碎片。

# 18. GIL

GIL 是 CPython 中的全局解释器锁。

核心影响：

- 同一时刻只有一个线程执行 Python 字节码。
- 因此 CPU 密集型任务，多线程通常无法真正利用多核。
- I/O 密集型任务，多线程仍然有用，因为线程在等待 I/O 时会释放 GIL。

应对方式：

- CPU 密集：多进程、C 扩展、NumPy、Cython
- I/O 密集：多线程、异步 I/O

注意：

- GIL 是 CPython 的实现细节，不是 Python 语言规范本身。

# 19. 多线程中的锁

你原文里提到"锁之类的东西"，这里补上常见高频点。

## 19.1 `Lock`

最基础的互斥锁，同一时刻只允许一个线程进入临界区。

## 19.2 `RLock`

可重入锁，同一线程可重复获取，适合递归或嵌套调用。

## 19.3 `Condition`

条件变量，用于线程间等待某个条件成立后继续执行。

## 19.4 `Semaphore`

信号量，用于限制并发数量。

## 19.5 `Event`

事件对象，线程可等待某个事件被设置。

## 19.6 `Barrier`

屏障，用于一组线程都到达某点后再统一继续。

# 20. 进程、线程、协程

## 20.1 进程

- 资源隔离强
- 稳定性高
- 开销比线程大
- 适合 CPU 密集或隔离要求高的场景

## 20.2 线程

- 同进程内共享内存
- 切换成本低于进程
- 适合 I/O 密集任务
- 受 GIL 影响

## 20.3 协程

- 用户态调度
- 切换开销更低
- 适合大量 I/O 并发
- 不是并行，通常仍在一个线程里调度

# 21. 进程间通信 IPC

常见方式：

- 管道 `Pipe`
- 队列 `Queue`
- 共享内存
- 信号量

- Socket

补充说明：

- `os.pipe()` 返回的是文件描述符，不是带 `send()` 方法的对象。
- 如果想方便地双向通信，常用 `multiprocessing.Pipe()` 。

# 22. `ASYNCIO` 重点

## 22.1 什么是 `asyncio`

`asyncio` 是 Python 标准库中的异步 I/O 框架，核心是事件循环。

## 22.2 协程

使用 `async def` 定义，调用返回协程对象。

## 22.3 事件循环

负责调度协程、任务、回调，管理 I/O 事件。

## 22.4 `create_task`

把协程包装成任务，交给事件循环并发执行。

## 22.5 `gather`

并发等待多个协程，并收集结果。

## 22.6 `Future` 和 `Task`

- `Future` 表示未来某个时刻才会拿到的结果。
- `Task` 是 `Future` 的子类，专门用于包装协程。

## 22.7 `run_in_executor`

异步代码中调用阻塞同步函数时，可放到线程池或进程池中执行，避免阻塞事件循环。

## 22.8 `asyncio.Queue`

注意：

- `asyncio.Queue` 是给协程用的。
- 它不是线程安全队列。
- 线程间通信通常用 `queue.Queue` 。
- 多进程通信通常用 `multiprocessing.Queue` 。

# 23. 模块、包、`__ALL__` 、DOCSTRING、类型注解

## 23.1 模块和包

- 一个 `.py` 文件通常可视为一个模块。
- 一个目录通常可视为一个包，现代 Python 里也存在命名空间包。

## 23.2 `__all__`

用于控制 `from module import *` 时导出的名字。

## 23.3 docstring

- 模块、类、函数、方法定义后的第一个字符串会成为 docstring。
- 可通过 `obj.__doc__` 或 `help(obj)` 查看。

## 23.4 类型注解

作用：

- 提高可读性
- 提升 IDE 补全和静态检查能力
- 配合 `mypy`、`pyright`、`pydantic` 等工具使用

示例：

```
def add(x: int, y: int) -> int:
    return x + y
```

注意：

- 类型注解默认不会在运行时强制校验。

# 24. 反射、猴子补丁、单例、多态、抽象类

## 24.1 反射

运行时动态获取或操作对象信息，常见函数：

- `getattr()`
- `setattr()`
- `hasattr()`
- `delattr()`
- `importlib.import_module()`

## 24.2 猴子补丁

在运行时替换模块、类、函数或方法。

用途：

- 打补丁
- 测试 mock
- gevent 对标准库 I/O 做协作式改造

风险：

- 隐式行为变化大
- 调试困难

## 24.3 单例模式

常见实现：

- 重写 `__new__`
- 模块级单例
- 装饰器
- 元类

## 24.4 多态

Python 更强调鸭子类型：

- 不关心对象“是不是某个类”
- 更关心对象“有没有这个方法”

## 24.5 抽象类和接口

Python 没有 Java 那种关键字级接口，通常用 `abc` 模块实现抽象基类。

# 25. PEP 8 和代码风格

PEP 8 是 Python 官方代码风格指南，常见考点：

- 4 空格缩进，不混用 Tab
- 类名用驼峰，函数和变量用蛇形命名
- 常量通常全大写
- 导入分组：标准库、第三方库、本地模块
- 每行长度适度控制
- 运算符、逗号、冒号周围空格风格统一

补充：

- Python 还有“禅宗”风格，即 `import this` 所展示的设计哲学。

# 26. OOM 和内存泄漏排查

## 26.1 常用观察方式

- `top` / `htop`
- `psutil`
- `tracemalloc`
- `objgraph`
- 系统日志

## 26.2 排查思路

1. 确认是不是 OOM 被系统杀死。
2. 查看容器或系统日志中是否存在 `Killed` 、 `out of memory` 。
3. 判断是内存持续上涨还是瞬时峰值过高。
4. 结合代码定位大对象、缓存、队列积压、无限列表增长等问题。

## 26.3 常见原因

- 大列表或字典不断增长
- 缓存未淘汰
- 消费速度跟不上生产速度
- 循环引用配合外部资源未释放
- C 扩展泄漏

# 27. 程序退出时内存是否会释放

一般情况下：

- 进程退出后，操作系统会回收该进程占用的绝大多数内存和文件描述符。

但仍需注意：

- C 扩展内部泄漏
- 共享内存段未正确 unlink
- 文件写缓冲尚未刷盘
- 数据库连接、socket、临时文件等资源应尽量显式关闭

所以最佳实践仍然是：

- 用 `with`

- 用 `try/finally`
- 必要时用 `atexit`

# 28. 递归

- Python 默认递归深度通常约为 1000。
- 可通过 `sys.getrecursionlimit()` 查看。
- 可通过 `sys.setrecursionlimit()` 修改。

注意：

- 递归过深可能导致栈溢出。
- Python 不做尾递归优化。

# 29. 常见零散高频点补充

## 29.1 `*args` 和 `**kwargs`

- `*args` 收集多余位置参数，得到元组。
- `**kwargs` 收集多余关键字参数，得到字典。

## 29.2 打乱列表

```
import random
nums = [1, 2, 3]
random.shuffle(nums)
```

`shuffle()` 原地修改列表，返回值是 `None`。

## 29.3 元组转字典

```
ks = (1, 2, 3)
vs = ("a", "b", "c")
print(dict(zip(ks, vs)))
```

## 29.4 `__all__`

只影响 `from xx import *`，不影响显式导入。

## 29.5 关键字补充

常见 Python 关键字包括：

- 流程控制：`if`、`elif`、`else`、`for`、`while`、`break`、`continue`、`pass`
- 异常处理：`try`、`except`、`finally`、`raise`
- 函数和类：`def`、`return`、`yield`、`lambda`、`class`
- 作用域：`global`、`nonlocal`
- 异步：`async`、`await`
- 逻辑：`and`、`or`、`not`、`is`、`in`

# 30. 面试时建议修正的几个表述

下面这些点如果按更严谨的说法回答，会更稳：

- Python 通常说是“强类型”，不建议说“弱类型”。
- `range()` 返回的是 `range` 对象，不是迭代器。
- `Template` 用的是 `substitute()`。
- `asyncio.Queue` 不是线程安全队列。
- 现代 Python 中 `dict` 已经保持插入顺序。

- 元组不是天然可哈希，前提是其元素都可哈希。
- Python 传参更准确的说法是“对象引用传递”。
- `super()` 是沿 MRO 查找，不只是“调用父类”。

# 31. 一段适合面试开场的总结

如果面试官让你概括 Python，你可以这样答：

Python 是一种解释型、动态类型、强类型的高级语言，语法简洁，开发效率高，支持面向对象、函数式和异步编程。它在 Web、爬虫、运维、数据分析和 AI 领域应用非常广。Python 的优势是生态成熟、表达力强、迭代快；劣势是原生执行效率相对一般，并且在 CPython 下受 GIL 影响，CPU 密集型任务通常更适合多进程或结合 C 扩展优化。

---

如果后续你要继续扩展，这份文档最适合再补三块：

1. Python 描述符、属性查找链、`__getattr__` / `__getattribute__`
2. Django / Flask / FastAPI 常见面试题
3. CPython 实现细节，如字节码、引用计数、对象模型